SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks:50**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Abarajithan S(192521299) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Abarajithan S

Annexure A

ANALYTICAL PROBLEM SOLVING

1 Trace how the arithmetic expression w = (a + b) * (c + d) - f is processed through the various phases of a compiler. Identify the tokens produced during lexical analysis, construct the corresponding syntax tree during parsing, generate the intermediate code using three-address representation, apply suitable code optimization techniques, and finally illustrate the possible target code. Explain how each phase contributes to efficient execution of the given expression.

2 Analyse the tokenization process of the expression float result = index----value + ++total; by identifying potential ambiguities in recognizing decrement and increment operators. Explain how a LEX-based lexical analyzer interprets such overlapping patterns, refine the lexical rules to correctly distinguish between operators, and justify how the modified specification ensures accurate and unambiguous token generation..

3 Determine the total number of tokens present in the following C program segment by identifying keywords, identifiers, operators, constants, and punctuation symbols, while ignoring whitespace and Additionally, analyze the following C program and identify all lexical errors. Explain why each error is invalid according to lexical rules:

```
int main()
{
 int a = 10, b - 30;
 if (ab)
   return(b):
 else
   return(a);
}
void main()
{
 int x = 10, y = 20;
 char * a;
 a = &x;
 x = Ixab;
}
```

4 Consider the regular expressions used for token identification, where an identifier is defined as [a-zA-Z_][a-zA-Z0-9_]* and an integer literal is represented by [0-9]. Using these

specifications, explain how the statement x = p + q * r / 2 (where x, p, q, and r are real variables) is processed through the various phases of a compiler. Describe how tokens are recognized during lexical analysis, how the structure is built during parsing, and how the expression is transformed into intermediate and target code during the overall translation process.

5 Construct suitable regular expressions for different subsets of binary strings over {0,1} by considering specific constraints: determine a regular expression for all strings that do not contain the substring 110, develop another expression for strings that include both 101 and 010 as substrings, and formulate a regular expression representing strings in which both the number of 0's and 1's are odd.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Compiler Phases for w = (a+b) * (c+d) - F

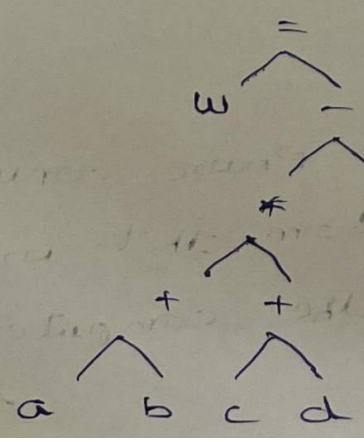
1. Lexical Analysis

The expression is divided into tokens.

w ⟶ identifier
= ⟶ Assignment operator
(⟶ left Paranthesis
a ⟶ identifier
+ ⟶ Addition operator
b ⟶ identifier
) ⟶ Right Paranthesis
* ⟶ multiplication operator
(⟶ Left Paranthesis
c ⟶ identifier
+ ⟶ Addition operator
d ⟶ identifier
) ⟶ Right Paranthesis
- ⟶ Subtraction operator
F ⟶ identifier

2. Syntax Analysis

The Syntax tree represents order of operations:



3. Three Address code

$t_1 = a + b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$t_4 = t_3 - F$

$w = t_4$

4. Code optimization

optimized code :

$t_1 = a + b$

$t_2 = c + d$

$t_1 = t_1 * t_2$

$w = t_1 - F$

5. Target code

```
Load R1, a
Add R1, b
Load R2, c
Add R2, d
Mul R1, R2
Sub R1, F
Store w, R1
```

Conclusion :

Each compiler phase converts the program into a form that is easier and faster for the computer to execute.

Q2. Tokenization of float result = index - - -
value + ++total ;

Tokens :

Float → keyboard
Result → identifier
= → Assignment operator
idex → identifier
- - → Decrement operator
- - → Decrement operator
value → identifier
+ → Addition operator
++ → increment operator
total → identifier
; → Punctuation

The important point is :

index - - - - value

can be divided as :

index - - - - value

Here, - - is recongnized as decrement operator.

LEX rule :

LEX follows the longest match rule. Therefore, when ++ or - - is available, it recognizes the complete operator instead of treating the symbols separately.

Example:

```
"++"   return INC;
"--"   return DEC;
"+"    return PLUS;
"-"    return MINUS;
```

Conclusion:

Using separate rules for ++, --, + and - makes token recognition clear and avoids ambiguity.

Q3. Token count and lexical errors

Program:

```
int main()
{
    int a = 10, b = 30;
    if (a<b)
        return (b);
    else
        return (a);
}
```

Tokens:

Ignoring whitespace:

int → keyword
main → identifier
() → Punctuation
int → keyword
a → identifier

Program 2 :

```
void main()
{
    int x = 10, y = 20;
    char *a;
    a = &x;
    x = Ixab;
}
```

Lexical observation :

Ixab is treated as an identifier, it is not a lexical error by itself.

However :

```
a = &x;
```

and the assignment to x may produce semantic / type errors, not lexical errors.

Q4. Compiler Processing of x = p + q * r / 2

The given identifier rule is

[a-zA-Z_][a-zA-Z0-9_]*

and integer literals are represented by [0-9].

1. Lexical Analysis :

x → identifier

= → Assignment

p → identifier

= → operator

10 → constant

, → punctuation

b → identifier

- → operator

30 → constant

if → keyword

a b → identifier

return → keyword

( ) → punctuation

b → identifier

: → Invalid punctuation

else → keyword

return → keyword

( ) → punctuation

a → identifier

; → punctuation

Errors:

(i) b - 30

It should Probably as:

b = 30

(ii) return (b):

It should be:

return (b);

+ → Addition

q → identifier

* → Multiplication

r → identifier

/ → Division

2 → Integer constant

2. Parsing:

Multiplication & division have higher priority than addition

Therefore:

x = p + ((q * r) / 2)

3. Intermediate code:

$t_1 = q * r$

$t_2 = t_1 / 2$

$t_3 = p + t_2$

$x = t_3$

4. Target code:

```
Load R1, q
MUL R1, r
DIV R1, 2
Load R2, p
Add R2, R1
store x, R2
```

Q5. Regular expressions

1. Strings that do not contain 110

A suitable expression is:

$(0|10)^*1^*$

This generates strings without the substring 110

2. Strings containing both 101 and 010

A simple expression is:

$(011)^*(101)(011)^*(010)(011)^*$

|

$(011)^*(010)(011)^*(101)(011)^*$

This allows either 101 to occur before 010 or 010 to occur before 101.

3. Strings having odd number of 0s and odd number of 1s

A suitable regular expression is:

$0(00)^*(11)^*1$

with appropriate interleaving needed for general strings.